

UNCOVERING HEALTHCARE PATTERNS AND PATIENT INSIGHTS THROUGH MYSQL DATA ANALYSIS

MARK ANTHONY A. BUNA

Data Analyst

PROBLEM STATEMENT

- Healthcare organizations handle large volumes of patient data, yet actionable insights are often hidden in scattered records. Without efficient analysis, trends in admissions, diagnoses, treatments, and billing go unnoticed. This limits the potential for improving operations and delivering better patient outcomes. This project uses MySQL to uncover key healthcare patterns by exploring patient demographics, hospital performance, doctor workloads, and insurance billing data.

OBJECTIVE

- To utilize MySQL for analyzing healthcare data, with the goal of identifying patient trends, optimizing hospital operations, and enhancing data-driven insights.
- Uncover insights into healthcare trends by tackling these analytical questions:
 - Basic Queries (Data Exploration)**
 - How many unique patients are recorded in the dataset?
 - Retrieve a list of distinct hospitals where patients were admitted.
 - Find all patients diagnosed with Diabetes, displaying their name, age, and hospital.
 - Count the number of male and female patients in the dataset.
 - What are the different types of admissions, and how many patients fall under each category?
 - Aggregation & Grouping**
 - Count the number of patients admitted per hospital, ordered by highest admissions.
 - What is the average billing amount for each admission type?

8. *Identify the hospital with the highest total billing amount across all admissions.*
9. *Retrieve the top 5 most common medical conditions, sorted by number of patients diagnosed.*
10. *Which doctor has treated the most patients, and how many have they treated?*
- **Date & Time Analysis**
 11. *What is the earliest and latest admission date recorded?*
 12. *Count the number of patients admitted in the last 30 days from the most recent admission date.*
 13. *Calculate the average length of stay for patients (difference between discharge and admission date).*
 14. *Find the month and year with the highest number of admissions and the corresponding count.*
 15. *Count how many emergency admissions happened on weekends.*
- **Advanced Queries (Joins, Subqueries, and Window Functions)**
 16. *Identify the most prescribed medication, along with the number of times it was prescribed.*
 17. *Retrieve details of patients with "Abnormal" test results, including their medical condition and assigned doctor.*
 18. *Which insurance provider has covered the highest total billing amount?*
 19. *Identify patients prescribed both "Aspirin" and "Ibuprofen" during their admission.*
 20. *Find the room number with the highest patient occupancy and how many patients stayed there.*



FINAL PORTFOLIO DELIVERABLES

- ✓ Built a MySQL database to analyze healthcare trends and billing, developed SQL queries to explore patient data and uncover insights, and documented the entire project in a GitHub repository.

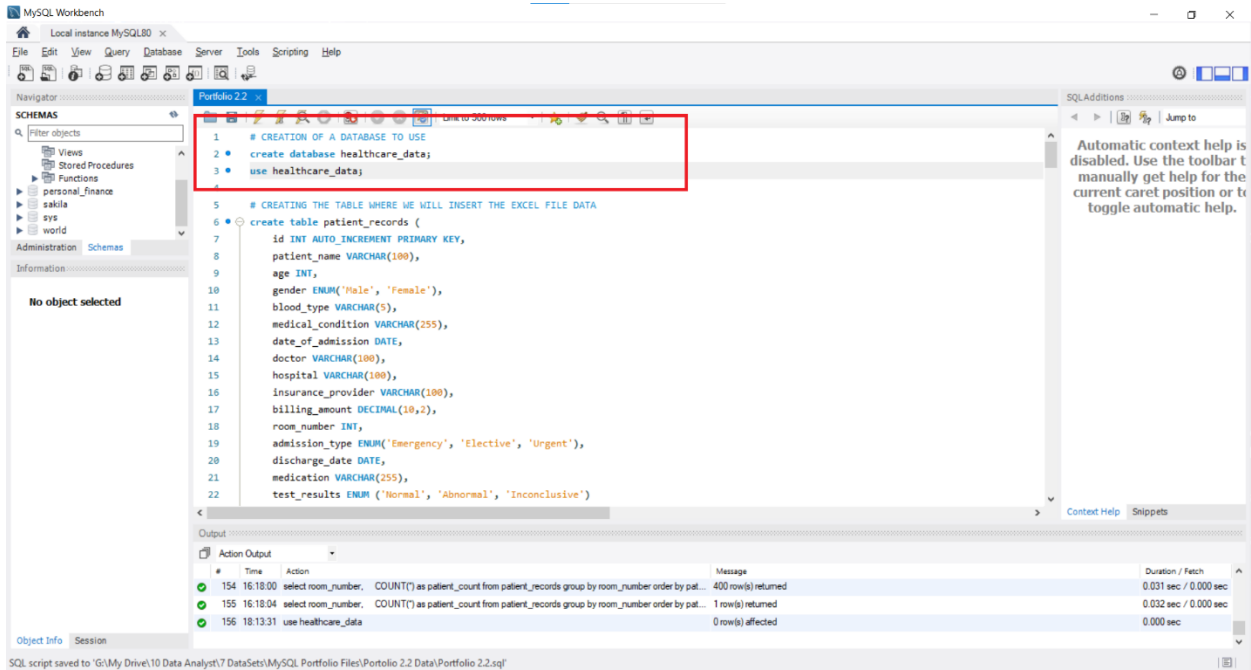


LINKS

- Dataset: <https://www.kaggle.com/datasets/parthgosavi/healthcare-dataset>
- Github: <https://github.com/marktheanalyst103/mysql-scripts/blob/main/Uncovering%20Healthcare%20Patterns%20and%20Patient%20Insights%20through%20Mysql%20Data%20Analysis.sql>

PROJECT STEPS

I. SET UP MySQL DATABASE



MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

Schemas

Filter objects

Views

Stored Procedures

Functions

personal_finance

sakila

sys

world

Administration Schemas

Information

No object selected

Portfolio 2.2 .x

```
1 # CREATION OF A DATABASE TO USE
2 create database healthcare_data;
3 use healthcare_data;
4
5 # CREATING THE TABLE WHERE WE WILL INSERT THE EXCEL FILE DATA
6 create table patient_records (
7 id INT AUTO_INCREMENT PRIMARY KEY,
8 patient_name VARCHAR(100),
9 age INT,
10 gender ENUM('Male', 'Female'),
11 blood_type VARCHAR(5),
12 medical_condition VARCHAR(255),
13 date_of_admission DATE,
14 doctor VARCHAR(100),
15 hospital VARCHAR(100),
16 insurance_provider VARCHAR(100),
17 billing_amount DECIMAL(10,2),
18 room_number INT,
19 admission_type ENUM('Emergency', 'Elective', 'Urgent'),
20 discharge_date DATE,
21 medication VARCHAR(255),
22 test_results ENUM('Normal', 'Abnormal', 'Inconclusive')
```

SQL Additions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

Output

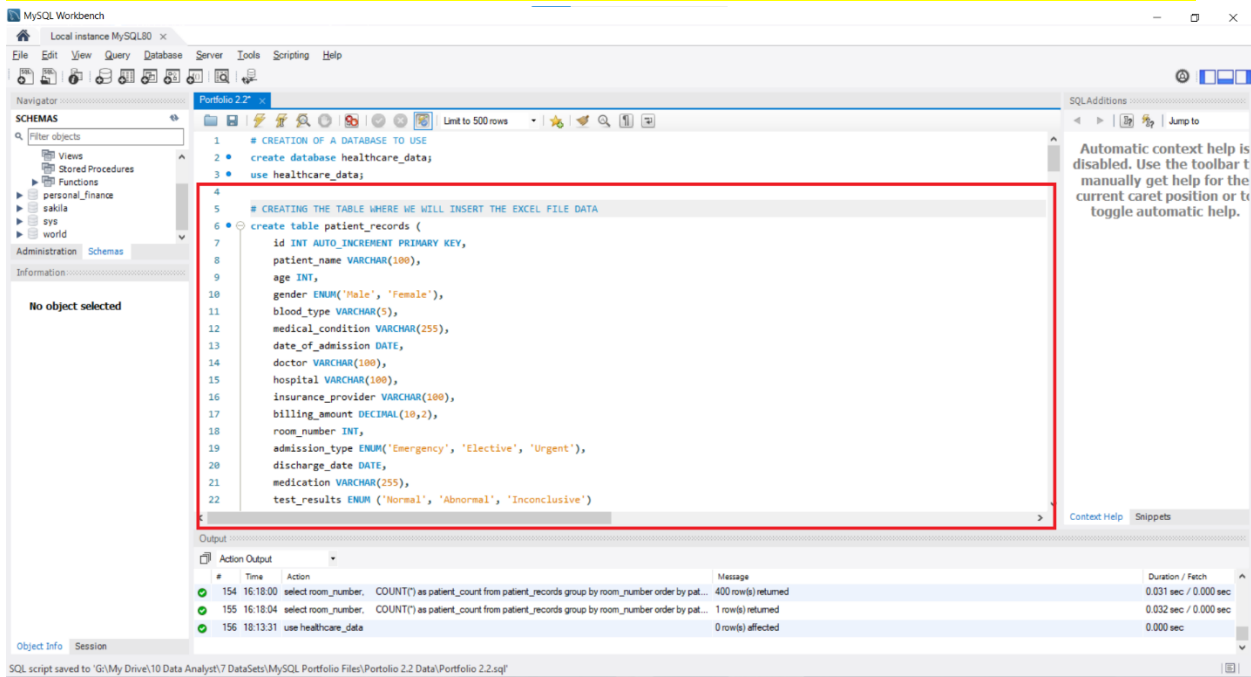
Action Output

#	Time	Action	Message	Duration / Fetch
154	16:18:00	select room_number, COUNT(*) as patient_count from patient_records group by room_number order by patient_count;	400 row(s) returned	0.031 sec / 0.000 sec
155	16:18:04	select room_number, COUNT(*) as patient_count from patient_records group by room_number order by patient_count;	1 row(s) returned	0.032 sec / 0.000 sec
156	18:13:31	use healthcare_data;	0 row(s) affected	0.000 sec

Object Info Session

SQL script saved to 'G:\My Drive\10 Data Analyst\7 DataSets\MySQL Portfolio Files\Portfolio 2.2 Data\Portfolio 2.2.sql'

II. PREPARATION OF THE MYSQL TABLE FOR INSERTING EXCEL FILE DATA



MySQL Workbench

Local instance MySQL80 x

File Edit View Query Database Server Tools Scripting Help

Navigator

Schemas

Filter objects

Views

Stored Procedures

Functions

personal_finance

sakila

sys

world

Administration Schemas

Information

No object selected

Portfolio 2.2 .x

```
1 # CREATION OF A DATABASE TO USE
2 create database healthcare_data;
3 use healthcare_data;
4
5 # CREATING THE TABLE WHERE WE WILL INSERT THE EXCEL FILE DATA
6 create table patient_records (
7 id INT AUTO_INCREMENT PRIMARY KEY,
8 patient_name VARCHAR(100),
9 age INT,
10 gender ENUM('Male', 'Female'),
11 blood_type VARCHAR(5),
12 medical_condition VARCHAR(255),
13 date_of_admission DATE,
14 doctor VARCHAR(100),
15 hospital VARCHAR(100),
16 insurance_provider VARCHAR(100),
17 billing_amount DECIMAL(10,2),
18 room_number INT,
19 admission_type ENUM('Emergency', 'Elective', 'Urgent'),
20 discharge_date DATE,
21 medication VARCHAR(255),
22 test_results ENUM('Normal', 'Abnormal', 'Inconclusive')
```

SQL Additions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

Output

Action Output

#	Time	Action	Message	Duration / Fetch
154	16:18:00	select room_number, COUNT(*) as patient_count from patient_records group by room_number order by patient_count;	400 row(s) returned	0.031 sec / 0.000 sec
155	16:18:04	select room_number, COUNT(*) as patient_count from patient_records group by room_number order by patient_count;	1 row(s) returned	0.032 sec / 0.000 sec
156	18:13:31	use healthcare_data;	0 row(s) affected	0.000 sec

Object Info Session

SQL script saved to 'G:\My Drive\10 Data Analyst\7 DataSets\MySQL Portfolio Files\Portfolio 2.2 Data\Portfolio 2.2.sql'

III. INSERTION OF EXCEL FILE DATA

patient_name	age	gender	blood_type	medical_condition	date_of_admission	doctor	hospital	insurance_provider	billing_amount	room_number	admission_type	discharge_date	medication	test_results
1 patient_name														
2 Bobby JacksOn	30	Male	B-	Cancer	1/31/2024	Matthew Smith	Sons and Miller	Blue Cross	18856.28131	328	Urgent	2/7/2024	Paracetamol	Normal
3 Leslie TerFly	62	Male	A+	Obesity	8/20/2019	Samantha Davies	Kim Inc	Medicare	33643.32729	265	Emergency	8/26/2019	Ibuprofen	Inconclusive
4 DaNiy sMiH	76	Female	A-	Obesity	9/22/2022	Tiffany Mitchell	Cook PLC	Aetna	27955.09608	205	Emergency	10/7/2022	Aspirin	Normal
5 andEw waTIS	28	Female	O+	Diabetes	11/18/2020	Kevin Wells	Hernandez Rogers and Vang,	Medicare	37909.78241	450	Elective	12/18/2020	Ibuprofen	Abnormal
6 adriENNE sEi	43	Female	AB+	Cancer	9/19/2022	Kathleen Hanna	White-White	Aetna	14238.81781	458	Urgent	10/9/2022	Penicillin	Abnormal
7 EMILY JOHNSOn	36	Male	A+	Asthma	12/20/2023	Taylor Newton	Nunes-Humphrey	UnitedHealthcare	48145.11095	389	Urgent	12/24/2023	Ibuprofen	Normal
8 edwArD EDwARds	21	Female	AB-	Diabetes	11/3/2020	Kelly Olson	Group Middleton	Medicare	19580.87234	389	Emergency	11/15/2020	Paracetamol	Inconclusive
9 ChrisTiNa MARTinez	20	Female	A+	Cancer	12/28/2021	Suzanne Thomas	Powell Robinson and Valdez,	Cigna	45820.46272	277	Emergency	1/7/2022	Paracetamol	Inconclusive
10 JASmiNe aGullaR	82	Male	AB+	Asthma	7/1/2020	Daniel Ferguson	Sons Rich and	Cigna	50119.22279	316	Elective	7/14/2020	Aspirin	Abnormal
11 CHRiStopheR BeRG	58	Female	AB-	Cancer	5/23/2021	Heather Day	Padilla-Walker	UnitedHealthcare	19784.63106	249	Elective	6/22/2021	Paracetamol	Inconclusive
12 miChElle danILes	72	Male	O+	Cancer	4/19/2020	John Duncan	Schaefer-Porter	Medicare	12576.79561	394	Urgent	4/22/2020	Paracetamol	Normal
13 aaRon MARTiNEZ	38	Female	A-	Hypertension	8/13/2023	Douglas Mayo	Lyons-Blair	Medicare	7999.58688	288	Urgent	9/5/2023	Liptor	Inconclusive
14 conOR HANsEn	75	Female	A+	Diabetes	12/12/2019	Kenneth Fletcher	Powers Miller, and Flores	Cigna	43282.28336	134	Emergency	12/18/2019	Penicillin	Abnormal
15 iObErT bauer	68	Female	AB+	Asthma	5/22/2020	Theresa Freeman	Rivera-Gutierrez	UnitedHealthcare	33207.70663	309	Urgent	6/19/2020	Liptor	Normal
16 bROOkE brady	44	Female	AB+	Cancer	10/8/2021	Roberta Stewart	Morris-Arellano	UnitedHealthcare	40701.59923	182	Urgent	10/13/2021	Paracetamol	Normal
17 MS. nAtalie gAMble	46	Female	AB-	Obesity	1/1/2023	Maria Dougherty	Cline-Williams	Blue Cross	12263.35743	465	Elective	1/11/2023	Aspirin	Inconclusive
18 haley perkins	63	Female	A+	Arthritis	6/23/2020	Erica Spencer	Cervantes-Wells	UnitedHealthcare	24499.8479	114	Elective	7/14/2020	Paracetamol	Normal
19 mRS. Jamit CAMPBELL	38	Male	AB-	Obesity	3/8/2020	Justin Kim	Torres, and Harrison Jones	Cigna	17440.46544	449	Urgent	4/7/2020	Paracetamol	Abnormal
20 LUKE BuTtEss	34	Female	A-	Hypertension	3/4/2021	Justin Moore Jr.	Houston PLC	Blue Cross	18843.02302	260	Elective	3/14/2021	Aspirin	Abnormal
21 dANIEL schmidt	63	Male	B+	Asthma	11/15/2022	Denise Galloway	Hammond Ltd	Cigna	23762.20358	465	Elective	11/22/2022	Penicillin	Normal
22 HANtHY burris	67	Female	A-	Asthma	6/28/2023	Krista Smith	Jones LLC	Blue Cross	4251488865	115	Elective	7/2/2023	Aspirin	Normal
23 CHRiStopheR BRiGHt	48	Male	B+	Asthma	1/21/2020	Gregory Smith	Williams-Davis	Aetna	17695.91162	295	Urgent	2/9/2020	Liptor	Normal
24 KathRin StewarT	58	Female	O+	Arthritis	5/12/2022	Vanessa Newton	Clark-Mayo	Aetna	5998.102908	327	Urgent	6/10/2022	Liptor	Inconclusive
25 dR. EiliEn thomPsOn	59	Male	A+	Asthma	8/2/2021	Donna Martinez MD	and Sons Smith	Aetna	25250.05243	119	Urgent	8/12/2021	Liptor	Inconclusive
26 PAUL HEndERsOn	72	Female	AB-	Hypertension	5/15/2020	Stephanie Kramer	Garner-Bowman	Medicare	33211.29542	109	Emergency	6/8/2020	Paracetamol	Inconclusive
27 PeTER fITzGERald	73	Male	AB+	Obesity	5/15/2020	Angela Contreras	Barnes-Bowman	Medicare	19746.83201	162	Urgent	5/20/2020	Aspirin	Abnormal
28 cathy sMall	51	Female	O-	Asthma	12/23/2023	Wendy Glen	Brown, and Jones Weaver	Blue Cross	26786.52956	401	Elective	1/19/2024	Ibuprofen	Normal
29 mR. KenNEth MoORE	34	Female	A+	Diabetes	6/21/2022	James Ellis	Serrano-Dixon	UnitedHealthcare	18834.80134	157	Emergency	6/30/2022	Liptor	Abnormal
30 MaRly HuNTER	38	Female	O-	Hypertension	1/3/2021	Jared Bruce Jr.	Gardner-Miller	Cigna	32643.29935	223	Emergency	1/16/2021	Penicillin	Normal
31 JOSHUA OLiver	63	Female	B+	Hypertension	10/3/2022	Brandy Mitchell	Guerrero-Boone	Aetna	5767.011054	293	Elective	10/12/2023	Paracetamol	Abnormal
32 THOMAS MARTiNEZ	34	Male	B-	Asthma	8/18/2019	Jacob Huynh	Hart Ltd	Cigna	47909.12881	371	Urgent	9/1/2019	Ibuprofen	Inconclusive
33 JANEs partERoon	23	Female	A+	Arthritis	11/19/2019	Kristina Frazier	Ortiz-Santiago	UnitedHealthcare	25835.32359	108	Urgent	11/29/2019	Penicillin	Abnormal
34 WILLIAM cOOPER	78	Male	AB-	Arthritis	5/18/2023	John Hartman	Group Duncan	Medicare	17993.2262	245	Elective	6/15/2023	Penicillin	Normal
35 Erin oRTEga	43	Male	AB-	Cancer	5/24/2023	Heather Garcia	Lopez-Phillips	Medicare	21185.95353	494	Elective	6/3/2023	Ibuprofen	Normal

- The data from the Microsoft Excel file that I need to upload and use for analysis in MySQL

```

17 billing_amount DECIMAL(10,2),
18 room_number INT,
19 admission_type ENUM('Emergency', 'Elective', 'Urgent'),
20 discharge_date DATE,
21 medication VARCHAR(255),
22 test_results ENUM('Normal', 'Abnormal', 'Inconclusive')
23 )
24 ;
25
26 select * from patient_records;
27
28 # ENABLE LOCAL INFILE VIA CONNECTION SETTINGS
29 SHOW VARIABLES LIKE 'local_infile'; -- This allows us to upload the Excel file
30 -- [It's ON already, but still not working...]
31 -- Turns out I had to use a Python script to fix this part
32
33
34 ## DATA CLEANING
35 # CLEANING THE NAMES (Capitalize the first letter of each word and remove titles such as Mr., Ms., Mrs., Dr., etc.)
36
37 DELIMITER $$
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

Output:

#	Time	Action	Message	Duration / Fetch
154	16:18:00	select room_number, COUNT(*) as patient_count from patient_records group by room_number order by pat...	400 row(s) returned	0.031 sec / 0.000 sec
155	16:18:04	select room_number, COUNT(*) as patient_count from patient_records group by room_number order by pat...	1 row(s) returned	0.032 sec / 0.000 sec
156	18:13:31	use healthcare_data	0 row(s) affected	0.000 sec

- For certain reasons, MySQL was not allowing me to upload the excel file so I had to find ways how to do it, and using python script is one that worked for me.

```
Jupyter MySQL (healthcare_data importation) Last Checkpoint: 24 days ago
File Edit View Run Kernel Settings Help Trusted
JupyterLab Python [conda env:base]

[5]: import pandas as pd
import mysql.connector

# 1. Load the CSV
file_path = "C:/Users/markb/Documents/healthcare_dataset.csv"
df = pd.read_csv(file_path)

# 2. Rename columns to match MySQL table
df.columns = [
    'patient_name', 'age', 'gender', 'blood_type', 'medical_condition',
    'date_of_admission', 'doctor', 'hospital', 'insurance_provider',
    'billing_amount', 'room_number', 'admission_type',
    'discharge_date', 'medication', 'test_results'
]

# 3. Convert date columns to YYYY-MM-DD format
date_cols = ['date_of_admission', 'discharge_date']
for col in date_cols:
    df[col] = pd.to_datetime(df[col], errors='coerce').dt.strftime('%Y-%m-%d')

# 4. Connect to MySQL
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="09273271736143#Abcd", # Replace with your actual MySQL password
    database="healthcare_data"
)
cursor = conn.cursor()

# 5. Insert each row
for index, row in df.iterrows():
    sql = """
    INSERT INTO patient_records
    (patient_name, age, gender, blood_type, medical_condition,
    date_of_admission, doctor, hospital, insurance_provider,
    billing_amount, room_number, admission_type,
    discharge_date, medication, test_results)
    VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
    """
    values = tuple(row)
    cursor.execute(sql, values)

# 6. Finalize
conn.commit()
cursor.close()
conn.close()

print("✅ Data imported successfully into patient_records.")
✅ Data imported successfully into patient_records.
```

- The Python script I used to upload the Excel file to MySQL for later analysis.

IV. DATA CLEANING

- Cleaning the names (Capitalize the first letter of each word and remove titles such as Mr., Ms., Mrs., Dr., etc.)

The screenshot shows the MySQL Workbench interface with a SQL script in the editor. The script is titled "Portfolio 2.2" and is used for cleaning names. It includes comments explaining the steps: capitalizing the first letter of each word and removing titles like Mr., Ms., Mrs., Dr., etc. The script uses a function named `TitleCase` to perform these operations. The output window shows the execution of the script, with a message indicating that 500 rows were returned.

```
33
34
35 -- DATA CLEANING
36 -- CLEANING THE NAMES (Capitalize the first letter of each word and remove titles such as Mr., Ms., Mrs., Dr., etc.)
37
38 DELIMITER $$
39
40 CREATE FUNCTION TitleCase(input TEXT) RETURNS TEXT -- 'input TEXT' is the messy name you give, and will be returned as a cleaned TEXT after
41 DETERMINISTIC -- ensures the function will always provide the same result for the same input
42 BEGIN
43     DECLARE result TEXT DEFAULT ''; -- "I want to make a box called 'result' where we'll store the name we're fixing, and right now it's empty
44     DECLARE word TEXT; -- "This is another box called 'word', which will hold individual words from the name we're fixing."
45
46     WHILE CHAR_LENGTH(input) > 0 DO -- "As long as there's something in the name (as long as it's not empty), keep going."
47         SET word = TRIM(SUBSTRING_INDEX(input, ' ', 1)); -- "Let's grab the first word in the name, and clean up any extra spaces around it."
48         -- (For example, if the name is " Mr. DAVID ", this grabs "Mr.")
49
50         IF CHAR_LENGTH(word) > 0 THEN -- "If the word is not empty, we proceed with formatting it."
51             SET result = CONCAT(
52                 result,
53                 UPPER(LEFT(word, 1)), -- "Take the first letter of the word and make it uppercase (e.g., 'm' becomes 'M')."
54                 LOWER(SUBSTRING(word, 2)), -- "Make the rest of the word lowercase (e.g., 'DAVID' becomes 'david')."
55                 ' ' -- Add a space after the word so that the next word can be processed later.
56             );
57         END IF;
58     END WHILE;
59
60     RETURN result;
61 END FUNCTION;
62
63 -- Finally, a cleaned and formatted patient_name
64 select * from patient_records;
65
```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

- Cleaning the Hospital Column (removing the "," at the end)

The screenshot shows the MySQL Workbench interface with a SQL script in the editor. The script is titled "Portfolio 2.2" and is used for cleaning the hospital column. It includes comments explaining the steps: removing the comma at the end of the hospital name. The script uses the `REPLACE` function to perform these operations. The output window shows the execution of the script, with a message indicating that 0 rows were affected.

```
94
95
96 -- Finally, a cleaned and formatted patient_name
97 select * from patient_records;
98
99 -- CLEANING THE HOSPITAL COLUMN (removing the "," at the end)
100
101 SELECT
102     hospital AS original_hospital,
103     REPLACE(hospital, ',', '') AS cleaned_hospital -- If there's a comma and maybe spaces at the end of the hospital name, just remove them
104 FROM patient_records
105
106
107 UPDATE patient_records
108 SET hospital = REPLACE(hospital, ',', '')
109
110
111 select * from patient_records; -- Hospital column cleaned successfully
112
```

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

id	patient_name	age	gender	blood_type	medical_condition	date_of_admission	doctor	hospital	insurance_provider	billing_amount
1	Bobby Jackson	30	Male	B-	Cancer	2024-01-31	Matthew Smith	Sons and Miller	Blue Cross	18856.28
2	Leslie Terry	62	Male	A+	Obesity	2019-08-20	Samantha Davies	Kim Inc	Medicare	33643.33
3	Danny Smith	76	Female	A-	Obesity	2022-09-22	Tiffany Mitchell	Cook PLC	Aetna	27955.10
4	Andrew Watts	28	Female	O+	Diabetes	2020-11-18	Kevin Wells	Hernandez Rogers and Vang	Medicare	37909.76
5	Adrienne Bell	43	Female	AB+	Cancer	2022-09-19	Kathleen Hanna	White-White	Aetna	14238.32

patient_records 124 x

Output

Action Output

158 18:38:45 use healthcare_data

0 row(s) affected

Duration / Fetch: 0.000 sec / 0.000 sec

V. MySQL QUERIES

Basic Queries (Data Exploration)

1. How many unique patients are recorded in the dataset?

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following queries:

```
-- BASIC QUERIES (Data Exploration)
#1. How many unique patients are recorded in the dataset?
select COUNT(patient_name) as unique_patients
from patient_records
;

-- or,
select COUNT(DISTINCT id) as unique_id
from patient_records
;

#2. Retrieve a list of distinct hospitals where patients were admitted.
select DISTINCT hospital as distinct_hospital
from patient_records
order by hospital -- not really necessary
;
```

The first query is highlighted with a red box. The Result Grid shows the output:

unique_patients
155500

The status bar at the bottom indicates: "159 18:40:15 select * from patient_records LIMIT 0, 500 500 row(s) returned".

2. Retrieve a list of distinct hospitals where patients were admitted.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following queries:

```
-- or,
select COUNT(DISTINCT id) as unique_id
from patient_records
;

#2. Retrieve a list of distinct hospitals where patients were admitted.
select DISTINCT hospital as distinct_hospital
from patient_records
order by hospital -- not really necessary
;

#3. Find all patients diagnosed with Diabetes, displaying their name, age, and hospital.
select
  patient_name,
  age,
  hospital
;
```

The second query is highlighted with a red box. The Result Grid shows the output:

distinct_hospital
Abbott and Thompson, Sullivan
Abbott Inc
Abbott Ltd
Abbott Moore and Williams
Abbott-Castillo
Abbott-Coleman

The status bar at the bottom indicates: "160 19:34:03 select COUNT(patient_name) as unique_patients from patient_records LIMIT 0, 500 1 row(s) returned".

3. Find all patients diagnosed with **Diabetes**, displaying their name, age, and hospital.

The screenshot shows the MySQL Workbench interface. The SQL editor contains two queries. The second query, highlighted with a red box, is:

```
#3. Find all patients diagnosed with Diabetes, displaying their name, age, and hospital.
select
  patient_name,
  age,
  hospital
from patient_records
where
  medical_condition LIKE '%Diabetes%' -- anything that contains the word 'Diabetes'
```

The Result Grid, also highlighted with a red box, displays the following data:

patient_name	age	hospital
Andrew Watts	28	Hernandez Rogers and Vang
Edward Edwards	21	Group Middleton
Connor Hansen	75	Powers Miller, and Flores
Mr. Kenneth Moore	34	Serrano-Dixon
Nicole Rodriguez	30	Poole Inc
Danise Torres	33	LLC Martin

The status bar at the bottom indicates "Query Completed".

4. Count the number of **male** and **female** patients in the dataset.

The screenshot shows the MySQL Workbench interface. The SQL editor contains two queries. The second query, highlighted with a red box, is:

```
#4. Count the number of male and female patients in the dataset.
select COUNT(gender) as unique_female
from patient_records
where
  gender LIKE 'Female'
```

The Result Grid, also highlighted with a red box, displays the following data:

unique_female
27726

The status bar at the bottom indicates "Query Completed".

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

Views
Stored Procedures
Functions
personal_finance
sakila
sys
world

Administration Schemas

Information

No object selected

Portfolio 2.2

Limit to 500 rows

```
136 }
137
138 #4. Count the number of male and female patients in the dataset.
139 select COUNT(gender) as unique_female
140 from patient_records
141 where
142     gender LIKE 'Female'
143 }
144
145 select COUNT(gender) as unique_male
146 from patient_records
147 where
148     gender LIKE 'Male'
149 }
150
151 #5. What are the different types of admissions, and how many patients fall under each category?
152 select
```

Result Grid

unique_male
27774

Result 129

Output

Action Output

#	Time	Action	Message	Duration / Fetch
163	19.37.56	select COUNT(gender) as unique_female from patient_records where gender LIKE 'Female' LIMIT 0, 500	1 row(s) returned	0.031 sec / 0.000 sec

Query Completed

5. What are the different types of admissions, and how many patients fall under each category?

MySQL Workbench

Local instance MySQL80

File Edit View Query Database Server Tools Scripting Help

Navigator

SCHEMAS

Filter objects

Views
Stored Procedures
Functions
personal_finance
sakila
sys
world

Administration Schemas

Information

No object selected

Portfolio 2.2

Limit to 500 rows

```
141 where
142     gender LIKE 'Female'
143 }
144
145 select COUNT(gender) as unique_male
146 from patient_records
147 where
148     gender LIKE 'Male'
149 }
150
151 #5. What are the different types of admissions, and how many patients fall under each category?
152 select
153     admission_type, COUNT(*) as patient_count
154 from patient_records
155 group by admission_type
156 }
157
```

Result Grid

admission_type	patient_count
Urgent	18576
Emergency	18269
Elective	18655

Result 130

Output

Action Output

#	Time	Action	Message	Duration / Fetch
164	19.38.19	select COUNT(gender) as unique_male from patient_records where gender LIKE 'Male' LIMIT 0, 500	1 row(s) returned	0.000 sec / 0.000 sec

Query Completed

Aggregation & Grouping

6. Count the number of **patients admitted per hospital**, ordered by the highest admissions.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
150
151 #5. What are the different types of admissions, and how many patients fall under each category?
152 select
153     admission_type, COUNT(*) as patient_count
154 from patient_records
155 group by admission_type
156
157
158 ## AGGREGATION & GROUPING
159 #6. Count the number of patients admitted per hospital, ordered by the highest admissions.
160 select
161     hospital, COUNT(*) as total_admission
162 from patient_records
163 group by hospital
164 order by total_admission desc
165
166
```

The query results are displayed in the Result Grid, showing the following data:

hospital	total_admission
LLC Smith	44
Ltd Smith	39
Johnson PLC	38
Smith Ltd	37
Smith Group	36
Smith PLC	36

The status bar at the bottom indicates that the query was completed successfully, returning 1 row(s).

7. What is the **average billing amount** for each admission type?

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
159 #6. Count the number of patients admitted per hospital, ordered by the highest admissions.
160 select
161     hospital, COUNT(*) as total_admission
162 from patient_records
163 group by hospital
164 order by total_admission desc
165
166
167 #7. What is the average billing amount for each admission type?
168 select
169     admission_type, ROUND(AVG(billing_amount),2) as avg_bill -- to display only two decimal places
170 from patient_records
171 group by admission_type
172
173
174 #8. Identify the hospital with the highest total billing amount across all admissions.
175 select
```

The query results are displayed in the Result Grid, showing the following data:

admission_type	avg_bill
Urgent	25517.36
Emergency	25497.40
Elective	25602.23

The status bar at the bottom indicates that the query was completed successfully, returning 500 row(s).

8. Identify the **hospital with the highest total billing amount** across all admissions.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
169 admission_type, ROUND(AVG(billing_amount),2) as avg_bill -- to display only two decimal places
170 from patient_records
171 group by admission_type
172 }
173
174 #8. Identify the hospital with the highest total billing amount across all admissions.
175 select
176 hospital, SUM(billing_amount) as total_bill
177 from patient_records
178 group by hospital
179 order by total_bill desc
180 limit 1
181 }
182
183 #9. Retrieve the top 5 most common medical conditions, sorted by the number of patients diagnosed.
184 select
185 medical_condition, COUNT(*) as patients_diagnosed
```

The query results are displayed in the Result Grid:

hospital	total_bill
Johnson PLC	1084202.70

The bottom status bar indicates: "168 19:40:51 select admission_type, ROUND(AVG(billing_amount),2) as avg_bill -- to display only two decimal places from ... 3 row(s) returned".

9. Retrieve the **top 5 most common medical conditions**, sorted by the number of patients diagnosed.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
175 select
176 hospital, SUM(billing_amount) as total_bill
177 from patient_records
178 group by hospital
179 order by total_bill desc
180 limit 1
181 }
182
183 #9. Retrieve the top 5 most common medical conditions, sorted by the number of patients diagnosed.
184 select
185 medical_condition, COUNT(*) as patients_diagnosed
186 from patient_records
187 group by medical_condition
188 order by patients_diagnosed desc
189 limit 5
190 }
```

The query results are displayed in the Result Grid:

medical_condition	patients_diagnosed
Arthritis	9308
Diabetes	9304
Hypertension	9245
Obesity	9231
Cancer	9227

The bottom status bar indicates: "169 19:41:25 select hospital, SUM(billing_amount) as total_bill from patient_records group by hospital order by total_bill desc... 1 row(s) returned".

10. Which doctor has treated the most patients, and how many have they treated?

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
185 medical_condition, COUNT(*) as patients_diagnosed
186 from patient_records
187 group by medical_condition
188 order by patients_diagnosed desc
189 limit 5
190 }
191
192 #10. Which doctor has treated the most patients, and how many have they treated?
193 select
194 doctor, COUNT(*) as patients_treated
195 from patient_records
196 group by doctor
197 order by patients_treated desc
198 limit 1
199
200
201 ## DATE & TIME ANALYSIS
```

The result grid shows the following data:

doctor	patients_treated
Michael Smith	27

The status bar indicates the query was completed successfully.

Date & Time Analysis

11. What is the earliest and latest admission date recorded?

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following queries:

```
195 from patient_records
196 group by doctor
197 order by patients_treated desc
198 limit 1
199
200
201 ## DATE & TIME ANALYSIS
202 #11. What is the earliest and latest admission date recorded?
203 select
204 MIN(date_of_admission) as earliest_admission,
205 MAX(discharge_date) as latest_admission
206 from patient_records
207
208
209 #12. Count the number of patients admitted in the last 30 days from the most recent admission date.
210 select
211 COUNT(*) as patients_last_30days
```

The result grid shows the following data:

earliest_admission	latest_admission
2019-05-08	2024-06-06

The status bar indicates the query was completed successfully.

12. Count the number of **patients admitted in the last 30 days** from the most recent admission date.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
205 MAX(discharge_date) as latest_admission
206 from patient_records
207 ;
208
209 #12. Count the number of patients admitted in the last 30 days from the most recent admission date.
210
211 select
212 COUNT(*) as patients_last_30days
213 from patient_records
214 where
215 date_of_admission >=
216 (
217 select MAX(date_of_admission) - INTERVAL 30 day
218 from patient_records
219 )
220 ;
221
222 #13. Calculate the average length of stay for patients (difference between discharge and admission date).
```

The query is executed, and the results are shown in the Result Grid:

patients_last_30days
1978

The status bar at the bottom indicates: "Query Completed".

13. Calculate the **average length of stay** for patients (difference between discharge and admission date).

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
210 select
211 COUNT(*) as patients_last_30days
212 from patient_records
213 where
214 date_of_admission >=
215 (
216 select MAX(date_of_admission) - INTERVAL 30 day
217 from patient_records
218 )
219 ;
220
221 #13. Calculate the average length of stay for patients (difference between discharge and admission date).
222 select
223 ROUND(AVG(DATEDIFF(discharge_date, date_of_admission)),2) as avg_length_of_stay
224 from patient_records
225 ;
226
```

The query is executed, and the results are shown in the Result Grid:

avg_length_of_stay
15.51

The status bar at the bottom indicates: "Query Completed".

14. Find the month and year with the highest number of admissions and the corresponding count.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
#14. Find the month and year with the highest number of admissions and the corresponding count.
select
  DATE_FORMAT(date_of_admission, '%Y-%M') as admission_month,
  COUNT(*) as admission_count
from patient_records
group by admission_month
order by admission_count desc
limit 1;
```

The query is executed, and the result is displayed in the Result Grid:

admission_month	admission_count
2020-August	1014

The status bar at the bottom indicates "Query Completed".

15. Count how many emergency admissions happened on weekends.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
#15. Count how many emergency admissions happened on weekends.
select
  COUNT(*) as emergency_weekend_admissions
from patient_records
where
  admission_type = 'Emergency'
and
  DAYOFWEEK(date_of_admission) IN (1,7) -- 1 means SUNDAY, 7 means SATURDAY;
```

The query is executed, and the result is displayed in the Result Grid:

emergency_weekend_admissions
5193

The status bar at the bottom indicates "Query Completed".

Advanced Queries (Joins, Subqueries, and Window Functions)

16. Identify the most prescribed medication, along with the number of times it was prescribed.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
-- ADVANCED QUERIES (JOINS, SUBQUERIES, AND WINDOW FUNCTIONS)
#16. Identify the most prescribed medication, along with the number of times it was prescribed.
select
    medication,
    COUNT(*) as prescription_count
from patient_records
group by medication
order by prescription_count desc
limit 1
```

The query is highlighted with a red box. Below the editor, the 'Result Grid' tab is active, showing the following result:

medication	prescription_count
Lipitor	11140

The result grid is also highlighted with a red box. The status bar at the bottom indicates 'Query Completed'.

17. Retrieve details of patients with "Abnormal" test results, including their medical condition and assigned doctor.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
#16. Identify the most prescribed medication, along with the number of times it was prescribed.
select
    medication,
    COUNT(*) as prescription_count
from patient_records
group by medication
order by prescription_count desc
limit 1

#17. Retrieve details of patients with "Abnormal" test results, including their medical condition and assigned doctor.
select *
from patient_records
where
    test_results = 'Abnormal'
```

The query is highlighted with a red box. Below the editor, the 'Result Grid' tab is active, showing the following result:

id	patient_name	age	gender	blood_type	medical_condition	date_of_admission	doctor	hospital	insurance_provider	billing_amount
4	Andrew Watts	28	Female	O+	Diabetes	2020-11-18	Kevin Wells	Hernandez Rogers and Vang	Medicare	37909.78
5	Adrienne Bell	43	Female	AB+	Cancer	2022-09-19	Kathleen Hanna	White-White	Aetna	14238.32
9	Jasmine Aguilar	82	Male	AB+	Asthma	2020-07-01	Daniel Ferguson	Sons Rich and	Cigna	50119.22
13	Connor Hansen	75	Female	A+	Diabetes	2019-12-12	Kenneth Fletcher	Powers Miller, and Flores	Cigna	43282.28
18	Mrs. Jamie Campbell	38	Male	AB-	Obesity	2020-03-08	Justin Kim	Torres, and Harrison Jones	Cigna	17440.47

The result grid is also highlighted with a red box. The status bar at the bottom indicates 'Query Completed'.

18. Which insurance provider has covered the highest total billing amount?

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
#17. Retrieve details of patients with "Abnormal" test results, including their medical condition and assigned doctor.
select *
from patient_records
where
test_results = 'Abnormal'
;

#18. Which insurance provider has covered the highest total billing amount?
select
insurance_provider,
SUM(billing_amount) as total_bill_covered
from patient_records
group by insurance_provider
order by total_bill_covered desc
limit 1
;
```

The query results are displayed in the Result Grid:

insurance_provider	total_bill_covered
Cigna	287139345.20

The status bar at the bottom indicates "Query Completed".

19. Identify patients prescribed both "Aspirin" and "Ibuprofen" during their admission.

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```

SUM(billing_amount) as total_bill_covered
from patient_records
group by insurance_provider
order by total_bill_covered desc
limit 1
;

#19. Identify patients prescribed both "Aspirin" and "Ibuprofen" during their admission.
select patient_name
from patient_records
where medication IN ('Aspirin', 'Ibuprofen')
group by patient_name
HAVING COUNT(DISTINCT medication) = 2 -- ensures both were prescribed
;

#20. Find the room number with the highest patient occupancy and how many patients stayed there.
select
;
```

The query results are displayed in the Result Grid:

patient_name
Aaron Lopez
Aaron Moore
Aaron Smith
Adam Gonzalez
Adam Hernandez
Alex Williams

The status bar at the bottom indicates "Query Completed".

20. Find the **room number with the highest patient occupancy** and how many patients stayed there.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
276 select patient_name
277 from patient_records
278 where medication IN ('Aspirin','Ibuprofen')
279 group by patient_name
280 HAVING COUNT(DISTINCT medication) = 2 -- ensures both were prescribed
281
282
283 #20. Find the room number with the highest patient occupancy and how many patients stayed there.
284 select
285     room_number,
286     COUNT(*) as patient_count
287 from patient_records
288 group by room_number
289 order by patient_count desc
290 limit 1
291
```

The result grid shows the following data:

room_number	patient_count
393	181

The bottom status bar indicates: Query Completed. 180 19:48:03 select patient_name from patient_records where medication IN ('Aspirin','Ibuprofen') group by patient_name H... 500 row(s) returned. Duration / Fetch: 0.032 sec / 0.000 sec.



KEY INSIGHTS

- The dataset includes **55,000 unique patients**, with a near-even gender split — **27,726 female patients**.
- **Admission types** are evenly distributed: **Urgent (18,576)**, **Emergency (18,269)**, **Elective (18,655)**.
- **LLC Smith Hospital** recorded the highest number of patient admissions (44), while **Johnson PLC** had the **highest total billing at ₱1,084,202.70**.
- Among **medical conditions**, the most common are **Arthritis (9,308)**, **Diabetes (9,304)**, and **Hypertension (9,245)**.
- **Michael Smith** stands out as the doctor with the most treated patients (27).
- Patients stay in the hospital for an **average of 15.51 days**, and **978 admissions** occurred in the last **30 days**.
- The **busiest month** was **August 2020** with **1,014 admissions**.
- **5,193 emergency admissions** took place during weekends, suggesting high weekend demand.
- The **most prescribed medication** appears over **11,140 times**, and **Cigna** was the top insurance provider by billing at **₱287.1M**.
- **Room 393** had the highest occupancy, accommodating **181 patients**.



RECOMMENDATIONS

- **Staffing & Resource Allocation:** Given the even distribution across admission types and frequent weekend emergencies, hospitals should ensure adequate weekend staffing and resources, especially for emergency cases.
- **Doctor Workload Monitoring:** With Michael Smith treating the most patients, reviewing doctor-patient ratios may help avoid potential burnout or improve efficiency.
- **Facility Usage:** High usage of Room 393 may point to availability issues or operational preference — deeper analysis could ensure better space optimization.
- **Medical Supply Planning:** Frequent prescriptions and top medical conditions (e.g., arthritis, diabetes) highlight areas to prioritize in medication inventory and chronic care programs.

- **Insurance & Billing Optimization:** Cigna's large billing coverage may indicate a strong partnership — but reviewing claim approval rates and processing times could help manage financial flows more effectively.

REFLECTION & NEXT STEPS

- This project sharpened my skills in writing complex SQL queries and analyzing real-world healthcare data. It gave me practical experience uncovering trends in patient care and billing. Moving forward, I plan to connect the data to a visualization tool to present insights more clearly and explore long-term patterns.